International Journal of Production Research

ISSN: (Print) (Online) Journal homepage: https://www.tandfonline.com/loi/tprs20

Multi-resource constrained dynamic workshop scheduling based on proximal policy optimisation

Peng Cheng Luo, Huan Qian Xiong, Bo Wen Zhang, Jie Yang Peng & Zhao Feng Xiong

To cite this article: Peng Cheng Luo, Huan Qian Xiong, Bo Wen Zhang, Jie Yang Peng & Zhao Feng Xiong (2021): Multi-resource constrained dynamic workshop scheduling based on proximal policy optimisation, International Journal of Production Research, DOI: 10.1080/00207543.2021.1975057

To link to this article: https://doi.org/10.1080/00207543.2021.1975057

Published online: 09 Sep 2021.

Submit your article to this journal

View related articles

View Crossmark data

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=tprs20

INTERNATIONAL JOURNAL OF PRODUCTION RESEARCH
https://doi.org/10.1080/00207543.2021.1975057

Taylor & Francis
Taylor & Francis Group

Check for updates

Multi-resource constrained dynamic workshop scheduling based on proximal policy optimisation

Peng Cheng Luo[a], Huan Qian Xiong[b], Bo Wen Zhang[a], Jie Yang Peng[a] and Zhao Feng Xiong [c]

[a]Advanced Manufacturing Technology Center, Tongji University, Shanghai, People's Republic of China; [b]Department of Electronic Science and Technology, Tongji University, Shanghai, People's Republic of China; [c]Shenzhen Leagsoft Technology Co., Ltd, Wuhan, Hubei Province, People's Republic of China

ABSTRACT

Multi-resource constrained dynamic workshop scheduling is a complex and challenging task in discrete manufacturing. In this paper, to obtain a high-performance scheduling in limited time, this problem is modelled into a Markov decision process, and solved by proximal policy optimisation algorithm, which can learn from the simulated workshop environment directly. A multi-modal hybrid neural network is used in the model to make good use of numerical state features representing workshop environment information and graphical state features representing constraint information during the learning process. Multi-label technique is used in this paper to decouple the output acts of jobs, machines, tools, and workers. Action mask technique coding the constraints is also used to prune invalid exploration. The experimental results show that compared with heuristic rules such as weighted shortest processing time, weighted modified due date, weighted cost over time, apparent tardiness cost and other reinforcement learning methods such as DeepRM and DeepRM2, the performance of the proposed method is at least 1.138% better in scheduling penalty.

ARTICLE HISTORY
Received 19 September 2020
Accepted 24 August 2021

KEYWORDS
Dynamic workshop scheduling; Markov decision process; multi-resource constrained scheduling; reinforcement learning; proximal policy optimisation

1. Introduction

For small and medium-sized manufacturers, whether jobs can be delivered in time directly affects their overall profitability and reputation. Basically, how to effectively arrange jobs to minimise the overall delay can be considered as a workshop scheduling problem. In actual production, the production resources including machines, tools and workers must be taken into account. However, the mutual coupling between different resources dramatically increases the complexity of the scheduling problem to solve. There are several factors that will limit the delivery of jobs: dynamic job arrivals, different job importance, limited due date and deadline, limited buffer, and the constraints on production resources. The implementation of reasonable workshop scheduling can bring three benefits:

- It can improve the production efficiency under the same production resources and environment;
- It can reduce the backlog of raw materials and unfinished jobs;
- It can improve the utilisation of production resources.

Therefore, it is of great practical significance to study the workshop scheduling problem for manufacturers.

Since workshop scheduling problems are generally NP-hard problems, it is difficult to obtain the exact solutions. However, these problems are still widely studied for their importance (Komaki, Sheikh, and Malakooti 2019; Yang and Xu 2021). Gao et al. (2019) made a comprehensive summary of papers using swarm intelligence (SI) and evolutionary algorithms (EA) in the research of flexible job shop problem (FJSP) in recent years. Touzout and Benyoucef (2019) compared repetitive single-unit process plan (RSUPP) algorithm, archive-based iterated local search (ABILS) algorithm, and non-dominated sorting genetic II algorithm (NSGA-II) under reconfigurable workshop environment. It is observed that ABILS algorithm has the characteristics of excluding bad solutions and generating a good process plan. Dou et al. (2020) proposed particle swarm optimisation (PSO) based crowding distance to optimise the total cost and tardiness in reconfigurable workshop scheduling problem. In order to maintain the diversity of solutions while converging to the optimal solution, both external

CONTACT Huan Qian Xiong 1832940@tongji.edu.cn

© 2021 Informa UK Limited, trading as Taylor & Francis Group

Pareto-optimal archive and roulette wheel selection are used. Though their PSO algorithm performs better in computation efficiency and scheduling solutions than NSGA-II, the high time complexity of their method is still a problem. Li et al. (2021) proposed a discrete artificial bee colony algorithm for distributed hybrid flowshop scheduling problem. To generate the initial solution, average factory assignment strategy and improved NEH algorithm are used. Their method is compared with other meta-heuristic algorithms through benchmarks and is proved to be useful in scheduling, however, their scheduling problem is still modelled as a static process. Though the application of meta-heuristic algorithms to scheduling problems has been well studied, in these works, constraints of resources such as worker, tool, and buffer are seldom considered, which makes it difficult to apply them to actual production.

In a resource-constrained scheduling problem, the application scenario is more complex because of additional constraints. Wang and Lu (2019) used a memorial algorithm with competition (MAC) to solve the capacitated green vehicle routing problem. Wang, Liu, and Zhou (2020) and others designed an improved bacterial forecasting algorithm in cellular manufacturing systems to solve the cell formation and product scheduling problem, in which the forgetting factors and worker constraints in practical problems are fully considered. Fu et al. (2019) used a global-local dual-population evolutionary optimisation algorithm to solve the dual-objective stochastic hybrid flow shop scheduling problem. They also used an information sharing mechanism to balance the algorithm's exploration and exploitation performance. Cao et al. (2018) adopted cuckoo search (CS) algorithm to the problem of semiconductor final testing scheduling problem, with consideration of sequence dependent setup time, operation sequence, and multi-resource requirement. Q-learning algorithm is used in their paper to optimise the parameter selection of CS algorithm, and support vector regression (SVR) is used to reduce the cost of computing time. As for the parallel batch processor scheduling problem of dynamically arriving jobs, Cao et al. (2020) proposed the surrogate assisted symmetric organisations search algorithm. The integration of SVR in Cao et al. (2018) and Cao et al. (2020) to fit the fitness function can reduce the time complexity of search algorithm and improve the response speed of scheduling system. For the FJSP problem, Cao contributed (Cao, Lin, and Zhou 2019) a knowledge-based cuckoo search algorithm, and combined it with SARSA algorithm to obtain the optimal parameters to generate new populations. The overall framework of the algorithm is still followed by using reinforcement learning (RL) algorithm to optimise the search algorithm parameters. Although some of the above algorithms try to integrate other models to reduce the performance cost, meta-heuristic algorithms still need to improve the initial solution through a lot of iterative calculations, which means it is hard for them to meet the time complexity required by actual production.

In a word, among these works in the field of scheduling, most meta-heuristic algorithms still model the scheduling process as a static problem and seldom consider the resource constraints. They cannot fully meet the requirements of dynamicity and flexibility in complex scheduling problems. Therefore, in actual multi-resource constrained scheduling, because of heuristic rules' characteristics of high efficiency and convenience (Oukil and El-Bouri 2021), schedulers usually design and use them to complete schedule, but the performance of a heuristic rule highly depends on the quality of it, and once the environment changes, the heuristic rules also need to be readjusted (Mao et al. 2016).

Reinforcement learning benefits from the rapid development of deep learning. Deep Q-learning network (DQN) is a groundbreaking work in the field of deep reinforcement learning (DRL) (Mnih et al. 2013), which uses a variant method of Q-learning to train a convolutional neural network (CNN), and it also uses experience replay to improve the learning performance of the model. Double DQN (Van Hasselt, Guez, and Silver 2016), priority replay buffer (Schaul et al. 2015), duelling DQN (Wang et al. 2016), distributional DQN (Bellemare, Dabney, and Munos 2017), and rainbow (Hessel et al. 2018) were proposed later to increase the training efficiency of the DQN model. These enhanced models also mitigate the problem of over estimation, and can provide a better model interpretation. Schulman and others proposed proximal policy optimisation (PPO) algorithm (Schulman et al. 2017) later. Compared to trust region policy optimisation (TRPO) algorithm (Schulman, Levine et al. 2015), PPO algorithm not only uses generalised advantage estimation (Schulman, Moritz et al. 2015) to replace long-term returns, but also simplifies objective functions and optimisation method. PPO has become a popular policy gradient (PG) algorithm since it has better performance than other PG algorithms such as A2C or A3C algorithm (Mnih et al. 2016), and it inherits the characteristics of the monotony increase from the TRPO, ensuring its stability in training process.

We believe that RL methods are well-suited to dynamic workshop scheduling because of their development in many other relevant fields. First, RL method models the scheduling process as a Markov decision process (MDP), and performs optimal scheduling in a sequential decision-making manner for dynamically arriving jobs, which is consistent with the dynamicity of

the problem itself. Second, the time complexity of RL methods is much lower than meta-heuristic algorithms. Third, RL methods can make good use of history statistics in the workshop, to achieve online or offline training. Finally, by continuously learning in a workshop environment, the RL policy can be optimised for a specific environment (e.g. different load, different number or kinds of production resources).

In fact, more researchers have been trying to introduce RL methods into the field of workshop scheduling, such as Q-learning (Watkins and Dayan 1992), REINFORCE (Sutton et al. 2000) and so on. Aydin used Q-III learning algorithm to train an agent to select scheduling rules dynamically (Aydin and Öztemel 2000). Their model uses the mean slack time and the length of job queue as system state, heuristic rules such as shortest processing time (SPT), cost over time (COVERT), and critical ratio (CR) as system action. In Aydin's experiments, the trained agent shows better scheduling performance than all rules they use as system action options. But in their work, the reward function is set according to the rule their model selects, thus the scheduling is still mainly based on prior knowledge, rather than the discovering mechanism of the agent. Wang, Usher, and others proposed a method using Q-learning algorithm for workshop resource scheduling problem (Wang and Usher 2005). Its objective function is to minimise maximum lateness, number of tardy jobs, and mean lateness. In Wei, and Zhao's work, the state is discretised through defining a production pressure, and they use a reward function defined by the average tardiness to train a Q-learning agent to select scheduling rules in a dynamic workshop (Wei and Zhao 2004). But only the discrete state features are used in their model, causing limited capability of information characterisation. Zhang, and Li utilised a gradient based value function approaching Q-learning algorithm to solve the dynamic weighted parallel machine problem (Zhang, Zheng, and Weng 2007). Weighted shortest processing time (WSPT), weighted modified due date (WMDD), weighted cost over time (WCOVERT) (Carrol 1965; Vepsalainen and Morton 1987), and ranking apparent tardiness cost with setups (RATCS) (Volgenant and Teerhuis 1999) are used as system actions, and abstract continuous features are used to describe the state of machines and jobs. Besides, the equality between their objective functions and rewards is proved in their paper. Later, Zhang used R-Learning (Mahadevan 1996) algorithm defining eight continuous features to solve the parallel machine problem with machine constraint (Zhang et al. 2012). Mao, Mohammad and others proposed DeepRM model to solve the scheduling problems with dynamic resource constraints, showing it is practical to use graphic features to describe the state of jobs (Mao et al. 2016). Ye, Ren and others introduced imitation learning and convolutional method to the DeepRM model, and successfully improved its performance, but they also found that REINFORCE algorithm will limit the overall performance of their model (Ye et al. 2018). Park and others used a PPO trained graph neural network to solve the job-shop scheduling problem (JSSP). The method proposed by them achieves a lower average makespan in benchmarks compared to various heuristic rules. However, the resource constraints in actual production are not fully considered in their work, and using heuristic rules in MDP as actions limits the job options during scheduling (Park et al. 2021).

After studying the potential of RL methods for workshop scheduling problem, this paper proposed a new model to solve multi-resource constrained dynamic workshop scheduling problem based on PPO. The performance of the model is evaluated through experiments and comparison with other algorithms. First, the dynamic scheduling problem is modelled into an MDP. After the state of the workshop is represented through multi-modal features, the equivalence of scheduling objective and reward function can be proved. The proposed model uses a multi-modal hybrid network to learn the scheduling policy and generate scheduling decisions. Action masks, multi-label techniques are used to optimise the robustness of PPO in our problem.

The rest of this paper is organised as follows. Section 2 describes the details of multi-resource constrained workshop scheduling problem, and relative settings are also given. Section 3 introduces the MDP transformed from the scheduling problem. In Section 4, the design of RL method used in this paper is introduced. All experimental results and comparisons with other algorithms are given in Section 5. Finally, Section 6 gives the conclusion of this paper.

2. Workshop scheduling problem

In a discrete workshop with single job and small batch production mode, a large number of overstocked buffer and the chaotic allocation of machines, tools, and workers could lead to low production efficiency and high production delay. In order to study and improve the management in small batch production, multi-resource constrained dynamic workshop scheduling is proposed in this paper to fully consider the resource constraints of tools, workers and buffer in discrete workshop production, which can be conducive to lean manufacturing.

Multi-resource constrained dynamic workshop scheduling is a modelling of scheduling problem in actual production. It can be described as:

- There are N independent jobs arriving the workshop according to a unknown distribution.
- Jobs will be selected from a queue of size K.
- The scheduling is constrained by M machines, I tools, and J workers.
- The objective is to minimise the tardiness penalty and expiration penalty.

2.1. The basic problem

Jobs arrive at the workshop dynamically. Arrived jobs will queue up, waiting to be scheduled. The maximum queue length is set to K. If the queue is full, new jobs will be rejected. Every job has a priority w, a process time p, a buffer usage u, a releasing time R, a delivery time E, and a deadline D once arrived. The time it will need for a job to leave workshop is L, which is decided by the production process.

2.2. Parallel dedicated machine

Parallel dedicated machine is a non-preemptive problem:

- Each job has only one process.
- Processing time is only related to the job itself.
- A job can only be processed on a set of designated machines.

2.3. The penalty of tardiness and expiration

Every job has a delivery time E and a deadline D. Jobs can only be delivered after the delivery time, and there will be a penalty based on the job's priority for every time step tardiness after the delivery time, which is defined as tardiness penalty in our paper. Furthermore, if the job cannot be completed before its deadline, it will expire. Expired jobs will be cancelled immediately, and there will be a full payment penalty in addition to the tardiness penalty. The extra full payment penalty is defined as expiration penalty.

The goal of scheduling is to minimise the sum of all penalties in one episode, and our objective function is shown below:

$$\textit{Minimize}\ \text{object} = \sum_{n=1}^{N} w_n[(L_n - E_n)^+ + \varphi_n p_n]$$

$$\varphi_n = \begin{cases} 1 \text{ if job } n \text{ is expired} \\ 0 \text{ if job } n \text{ is not expired} \end{cases}$$

$$L_n = \begin{cases} D_n \text{ if job } n \text{ is expired} \\ C_n \text{ if job } n \text{ is not expired} \end{cases} \tag{1}$$

where

$$(L_n - E_n)^+ \text{ means } \max\{L_n - E_n, 0\} \tag{2}$$

In (1), N means the number of jobs scheduled, and L_n means the time when job n leaves the system. C_n is the complete time of job n, and φ is the indicating function of job n's cancelling state. For any job not completed by the deadline, L_n equals its deadline. $w_n(L_n - E_n)^+$ is the tardiness penalty of job n. $w_n\varphi_n p_n$ is expiration penalty of job n.

2.4. Dedicated tools and workers

Tools and workers are non-preemptive resources in our problem. Each machine will need one worker and one tool to complete a process. A machine can only use a set of dedicated tools and workers, and the relationship between them is static and determined by production workshop environment.

2.5. The constraint of buffer

The workshop environment has a buffer with a maximum capacity. Every complete job or job being processed will take up a certain space of buffer until its delivery. The buffer constraint ensures that no complete jobs will cause jam in the production, any job that will take more than the spare space of the buffer will not be scheduled.

3. The MDP model for workshop scheduling

There is an internal clock in the scheduling system. The system clock uses an integer to define the current time (τ) of the system. This integer will update with the production process, and will be recovered to 1 when an episode ends and the system resets. While clock steps, the state of the workshop will change, and the state transition may include incidents such as arrival of new jobs or releasing of resources. While the number of jobs scheduled by the system reaches a preset maximum jobs limit N, the current episode will end.

Every scheduling decision has four acts, and there is a cascade relation between each act:

- First, choose a job to process.
- Second, choose one of the available machines according to this job.
- Finally, choose a tool and a worker according to the job and the machine we choose.

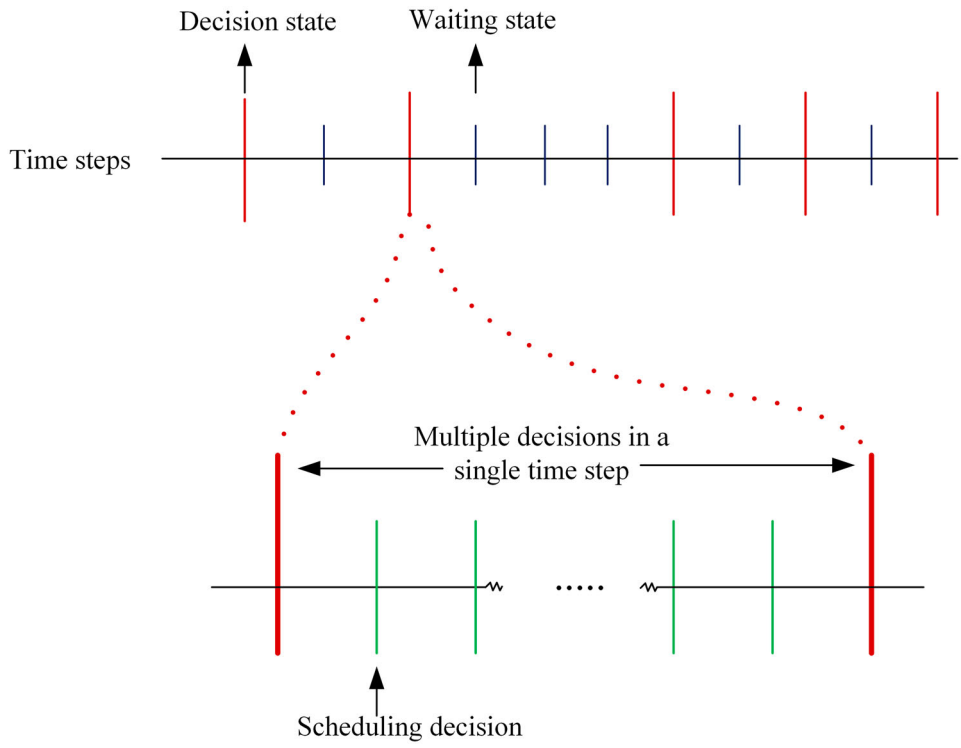

Figure 1. The time steps and state transition of our workshop environment.

Every decision should meet its constraints of acts. If a decision does not meet constraints, the decision becomes an invalid decision which will not influence the workshop environment. If the decision meets all constraints, we call the decision a valid decision.

For every state, if any valid decision could be made, it becomes a decision state, otherwise we define it as a waiting state. For a decision state, there is one output scheduling decision, setting one job in process with all necessary resources, then the system will step to next state without clock step, as is shown in Figure 1. For a waiting state, the system will only be updated by new events, and it will step to next state with a clock step directly.

3.1. State features

The production scheduler observes information of the current observation o_t, past observations $o_{t-1}, o_{t-2}, \ldots$ and past actions $a_{t-1}, a_{t-2}, \ldots$. We therefore use sequences of actions and observations as state $s_t = o_1, a_1, o_2, \ldots, a_{t-1}, o_t$. The model is designed to learn scheduling strategies that depend upon these sequences. All sequences in the workshop are assumed to terminate in a finite number of timesteps. This formalism gives rise to a large but finite MDP in which each sequence is a distinct state. As a result, we can apply standard reinforcement learning methods to MDPs, simply by using the complete sequence as the state representation s_t at step t. According to the definition of state features (Mnih et al. 2013), the Markov property can be easily proven:

$$P\left(S_{t+1} = s_{t+1} \mid S_{0:t} = s_{0:t}\right) = P\left(S_{t+1} = s_{t+1} \mid S_t = s_t\right) \tag{3}$$

Though sequences of actions and observations are chosen as state features in this paper, they are too complex and redundant for the algorithm. It is more practical to use refined features, for example, a combination of current observation frame and several past observation frames (Mnih et al. 2015). In this paper, the refined state feature consists of numerical state feature and graphical state feature. The graphical state feature is composed of the current frame and the past three frames.

3.1.1. Numerical features

We use both numerical and graphical features to encode the workshop's state. Numerical features are used to describe the state of machines, queue, buffer and other information.

Machine features are used to describe the state of each machine. $f_{m,c}$ means the cth feature of machine m. Queue features are used to describe each position in the queue, and $f_{l,c}$ means the cth feature of queue position l. Buffer features are used to describe each job in the buffer waiting to be delivered, and $f_{h,c}$ means the cth feature of buffer

position h. Common features are used to describe information about the whole system, and we use f_c as the mark of the cth feature.

- Machine feature 1 ($f_{m,1} = 1$ or 0): A Boolean feature describes whether the machine m idles. 1 means the machine m idles, and 0 means it is busy.
- Machine feature 2 ($f_{m,2} = w_n$): The priority of the job being processed by machine m. 0 means no job.
- Machine feature 3 ($f_{m,3} = p_n + S_n - \tau$): The remaining process time of the job being processed by machine m. S_n is the time when the job process begins.
- Machine feature 4 ($f_{m,4} = (E_n - \tau)^+$): Time left before the job being processed by machine m reaches the delivery time.
- Machine feature 5 ($f_{m,5} = 1$ or 0): A Boolean feature describes whether the job being processed by machine m is a tardy job. $f_{m,5} = 1$ means it is tardy. $f_{m,5} = 0$ means it is not tardy yet.
- Machine feature 6 ($f_{m,6} = (D_n - \tau)^+$): Time left before the job being processed by machine m reaches the deadline.
- Machine feature 7 ($f_{m,7} = p_n$): The process time of the job being processed by machine m.
- Machine feature 8 ($f_{m,8} = u_n$): The buffer usage of the job being processed by machine m.
- Queue feature 1 ($f_{l,1} = 1$ or 0): A Boolean feature describes whether position l of the queue is filled with a job. 0 means no job in position l.
- Queue feature 2 ($f_{l,2} = w_n$): The priority of the job in position l. If there is no job in position l, it will be 0.
- Queue feature 3 ($f_{l,3} = p_n$): The process time of the job in position l. If there is no job in position l, it will be 0.
- Queue feature 4 ($f_{l,4} = (E_n - \tau)^+$): Time left before the job in position l reaches the delivery time.
- Queue feature 5 ($f_{l,5} = 1$ or 0): A Boolean feature describes whether the job in position l is a tardy job. $f_{l,5} = 1$ means it is tardy. $f_{l,5} = 0$ means it is not tardy yet.
- Queue feature 6 ($f_{l,6} = -\frac{(E_n - p_n - \tau)^+}{p_n}$): Slack coefficient of the job in position l.
- Queue feature 7 ($f_{l,7} = -\frac{w_n}{p_n}$): WSPT rule coefficient of the job in position l.
- Queue feature 8 ($f_{l,8} = -\frac{\max\{p_n, E_n - \tau\}}{w_n}$): WMDD rule coefficient of the job in position l.
- Queue feature 9 ($f_{l,9} = \frac{w_n}{p_n} \exp(-\frac{(E_n - p_n - \tau)^+}{p_n h_{ATC}})$): Apparent tardiness cost (ATC) rule coefficient of the job in position l, where h_{ATC} is a hyperparameter of ATC rule.
- Queue feature 10 ($f_{l,10} = \frac{w_n}{p_n}[1 - \frac{(E_n - p_n - \tau)^+}{p_n K_{WCO}}]^+$): WCOVERT rule coefficient of the job in position l, where K_{WCO} is a hyperparameter of WCOVERT rule.
- Queue feature 11 ($f_{l,11} = (D_n - \tau)^+$): Time left before the job in position l reaches the deadline.
- Queue feature 12 ($f_{l,12} = (D_n - \tau)^+ - p_n$): The difference between $f_{l,11}$ and the job's process time.
- Queue feature 13 ($f_{l,13} = \text{sign}((D_n - \tau)^+ - p_n)$): A feature that indicates whether the job will be expired. $f_{l,13} = -1$ means the job cannot be completed before its deadline. $f_{l,13} = 0$ means the job will be expired unless it is chosen in the current time step. And $f_{l,13} = 1$ means the job still has time before being expired.
- Queue feature 14 ($f_{l,14} = p_n \times w_n$): The product of the process time and priority of the job in position j, which will be the expiration penalty if it is expired.
- Queue feature 15 ($f_{l,15} = 1$ or 0): This feature also indicates the expiration state of the job in position l. $f_{l,15} = 1$ while $(D_n - \tau)^+ > p_n$, and $f_{l,15} = 0$ if $(D_n - \tau)^+ < p_n$ or there is no job in position l.
- Queue feature 16 ($f_{l,16} = u_n$): The buffer usage of the job in position l.
- Queue feature 17 ($f_{l,17} = 1$ or 0): A Boolean feature describes whether the job in position l meets buffer constraint. If the job's buffer usage is smaller than the rest buffer of the workshop environment, $f_{l,17} = 1$. And $f_{l,17} = 0$ when the job's buffer usage is larger than the rest buffer or there is no job in position l.
- Queue feature 18 ($f_{l,18} = 1$ or 0): This is a Boolean feature that indicates both expiration state and buffer constraints. $f_{l,18} = 1$ only when $f_{l,15} = 1$ and $f_{l,17} = 1$.
- Buffer feature 1 ($f_{h,1} = u_n$ or 0): This feature indicates the buffer usage of the job in buffer position h. 0 means no job.
- Buffer feature 2 ($f_{h,1} = E_n - \tau$ or 0): This feature indicates how much time left before the job in buffer position h is delivered. 0 means no job.
- Common feature 1 ($f_1 = b(\tau)$): This feature is the rest buffer of the workshop environment.
- Common feature 2 ($f_2 = \tau - \tau_{last}$): This feature is the time difference between current state and the state of the last decision.

Since we set some state redundancy, there are numerical features that indicate same information but in a different way. Redundancy can be considered as what is called intermediate concept in a normal learning process, which can reduce the difficulty of learning tasks. For example, we use some Boolean features and rule coefficient features such as ATC and WCOVERT rule coefficients to help our model learn faster.

3.1.2. Graphical features

Graphical features are used to indicate the dynamic constraints of machine, tools, and workers, as is shown in Figure 2. The graphic has three area. The first area is used to describe the constraints of machine, followed by an area describing the constraints of tools. The last area is used to describe the constraints of workers. In the graphic, a white block means the constraint is met; a grey block means the resource is in use; a black block means no constraint.

In the first area, every row represents one position in the queue, and every column represents a machine. While a job is ready to be processed, the blocks representing available machines in the job's row will be white. If there is no job in the position, the corresponding row will be black. While a machine is in use, the blocks in its column will be grey.

In the second and third area, every row represents one tool or one worker, and every column represents a machine. The black blocks in these areas are fixed. When a tool or a worker is in use, only the block corresponding to the machine occupying it will be grey. This is because if we render the whole row grey, the information about which machine is using this resource will be lost.

In order to include historical scheduling information, we stack the past three frames of graphical features together with the current frame as the current graphical state feature. Actually, if we used the numerical features to encode the constraint relations, the vectors would be sparse. On the other hand, graphic method is inconvenient to encode the basic numerical information about our environment. Thus the combination of two methods can better help our model to understand the state features and extract deeper information.

3.2. Multi-action outputs

Scheduling model outputs four sub-actions sequentially in one decision:

- Act 1 (job action): Choose a job from K queue positions. Output value range: 0, 1,…, $K-1$.
- Act 2 (machine action): Choose a machine from M machines for process. Output value range: 0, 1,…, $M-1$.
- Act 3 (tool action): Choose a tool from I tools for process. Output value range: 0, 1,…, $I-1$.
- Act 4 (worker action): Choose a worker from J workers for process. Output value range: 0, 1,…, $J-1$.

Besides, if the scheduling model interacts with the environment through scheduling decisions directly, it may output lots of invalid decisions that do not satisfy the constraints, causing ineffective training. To solve this problem, we use action masks to encode the constraints of decisions. Action masks can block invalid actions according to the decision constraints. Action mask for every action branch is a vector, but since there is cascade relation between different action branches, we cannot determine the action mask of one branch before the model outputs all previous actions. One method to realise this function is to communicate with the environment

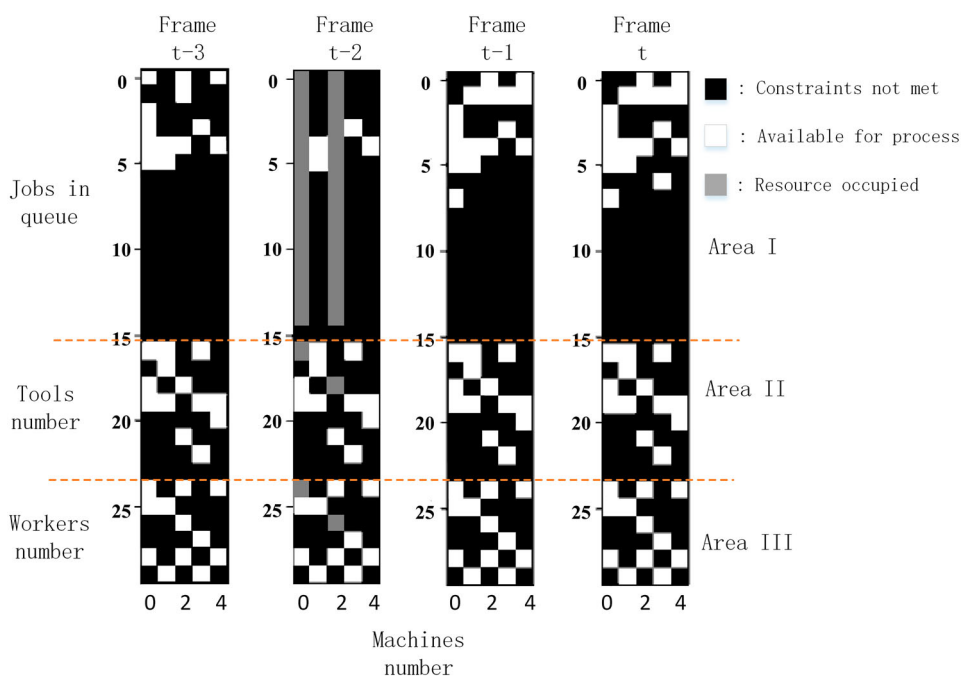

Figure 2. Three samples of graphical features.

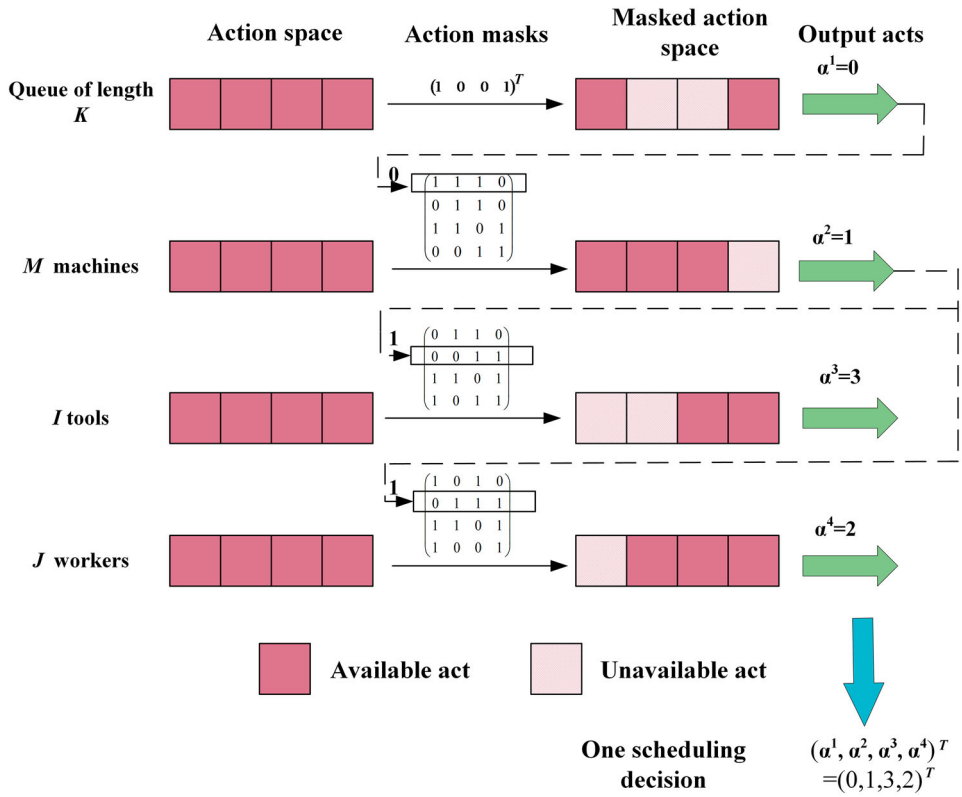

Figure 3. The process of making one output decision.

for multiple times, which is bad for code packaging and execution.

To solve this problem, we extract a matrix of action mask that contains all possible vectors for every branch from the environment and send it to the model along with the current state. Once all previous action decisions are made, the model will select the corresponding column as its action mask vector, as is shown in Figure 3. For example, once the job in queue position j is chosen to be processed, our model will choose the jth row of the action mask matrix as the input action mask. The definition of every action mask is shown below:

- Act 1 (job action): The action mask of Act 1 is a vector, in which each element corresponds to a position in the queue.
- Act 2 (machine action): The action mask of Act 2 is a two-dimensional matrix. Each row corresponds to a position in the queue. Each column corresponds to a machine.
- Act 3 (tool action): The action mask of Act 3 is a two-dimensional matrix. Each row corresponds to a machine. Each column corresponds to a tool.
- Act 4 (worker action): The action mask of Act 4 is a two-dimensional matrix. Each row corresponds to a machine. Each column corresponds to a worker.

The extra constraint that comes from dedicated production resources is also coded in action masks, in addition to constraints about non-preemption, dedicated relationship, and queue. For example, if all dedicated tools for a machine are occupied, the machine cannot be scheduled too.

3.3. Reward functions

Since the goal of a reinforcement learning is to maximise the reward function, the rationality of the reward function has great influence on whether the model could learn positive acts from its environment. In this paper, the reward function is equal to the objective function mentioned above so the consistency of optimisation goals can be ensured. The reward function mainly consists of tardiness part and expiration part.

The object function of tardiness penalty is (4):

$$\min \text{object tardiness} = \min \sum_{n=1}^{N(\tau)} w_n (L_n - E_n)^+ \quad (4)$$

where τ is the current time step, and $N(\tau)$ is the cumulative number of jobs arriving at the system until time step τ.

Then we define r_t^d as the tardiness reward between state s_{t-1} and s_t. In (5) and (6), τ_t means the time step

when the tth decision is made, and $\sigma_n(v)$ is the indicating function about job n.

$$r_t^d = \sum_{n=1}^{N(\tau)} \int_{\tau_{t-1}}^{\tau_t} w_n \sigma_n(v)\, \mathrm{d}v \tag{5}$$

$$\sigma_n(v) = \begin{cases} 0 & R_n \le v < \min\{L_n, E_n\} \\ -1 & \min\{L_n, E_n\} \le v \le L_n \end{cases} \tag{6}$$

Because

$$\begin{aligned} \lim_{\tau\to\infty} \sum_{t=1}^{NT(\tau)} r_t^d &= \lim_{\tau\to\infty} \sum_{t=1}^{NT(\tau)} \sum_{n=1}^{N(\tau)} \int_{\tau_{t-1}}^{\tau_t} w_n \sigma_n(v)\, \mathrm{d}v \\ &= \lim_{\tau\to\infty} \sum_{n=1}^{N(\tau)} \sum_{t=1}^{NT(\tau)} \int_{\tau_{t-1}}^{\tau_t} w_n \sigma_n(v)\, \mathrm{d}v \\ &= \lim_{\tau\to\infty} \sum_{n=1}^{N(\tau)} \int_{1}^{\tau} w_n \sigma_n(v)\, \mathrm{d}v \end{aligned} \tag{7}$$

where $NT(\tau)$ is the number of state transitions by time τ. Assume that

$$\begin{aligned} SJ1 &= \{n \mid L_n \ge E_n, n \le N(\tau)\} \\ SJ2 &= \{n \mid L_n < E_n, n \le N(\tau)\} \end{aligned} \tag{8}$$

then we have

$$\begin{aligned} &\lim_{\tau\to\infty} \sum_{t=1}^{NT(\tau)} r_t^d \\ &= \lim_{\tau\to\infty} \left(\sum_{n\in SJ1} \left[\int_{R_n}^{E_n} w_n \sigma_n(v)\, \mathrm{d}v + \int_{E_n}^{L_n} w_n \sigma_n(v)\, \mathrm{d}v \right] \right. \\ &\quad \left. + \sum_{n\in SJ2} \int_{L_n}^{E_n} w_n \sigma_n(v)\, \mathrm{d}v \right) \\ &= \lim_{\tau\to\infty} \left(\sum_{n\in SJ1} \left(0 + \int_{E_n}^{L_n} -w_n\, \mathrm{d}v \right) + \sum_{n\in SJ2} 0 \right) \\ &= \lim_{\tau\to\infty} \sum_{n\in SJ1} -w_n (L_n - E_n) \end{aligned} \tag{9}$$

Meanwhile,

$$\begin{aligned} &\lim_{\tau\to\infty} \text{object tardiness} \\ &= \lim_{\tau\to\infty} \sum_{n=1}^{N(\tau)} w_n (L_n - E_n)^+ \\ &= \lim_{\tau\to\infty} \sum_{n\in SJ1} w_n (L_n - E_n)^+ + \sum_{n\in SJ2} w_n (L_n - E_n)^+ \\ &= \lim_{\tau\to\infty} \sum_{n\in SJ1} w_n (L_n - E_n)^+ \end{aligned} \tag{10}$$

and

$$\min \lim_{\tau\to\infty} \text{object tardiness} \Leftrightarrow \max \lim_{\tau\to\infty} \sum_{t=1}^{NT(\tau)} r_t^d \tag{11}$$

Since $\sigma_n(v)$ is a function defined within $[R_n, L_n]$, we have

$$\begin{aligned} r_t^d &= \sum_{n=1}^{N(\tau)} \int_{\tau_{t-1}}^{\tau_t} w_n \sigma_n(v)\, \mathrm{d}v \\ &= \sum_{n\in SY(\tau_{t-1},\tau_t)} \int_{\tau_{t-1}}^{\tau_t} w_n \sigma_n(v)\, \mathrm{d}v \end{aligned} \tag{12}$$

where $SY(\tau_{t-1}, \tau_t)$ is the collection of jobs in scheduling system from time step τ_{t-1} to τ_t. The objective function of expiration is (13):

$$\min \text{object expired} = \min \sum_{n=1}^{N(\tau)} \varphi_n p_n w_n \tag{13}$$

We define r_t^e as the cumulative expiration reward from state s_{t-1} to s_t, and $\delta_n(v)$ as the indicating function II about job n:

$$r_t^e = \sum_{n=1}^{N(\tau)} \int_{\tau_{t-1}}^{\tau_t} w_n p_n \delta_n(v)\, \mathrm{d}v \tag{14}$$

$$\delta_n(v) = \begin{cases} 0 & R_n \le v \le \min\{D_n - 1, L_n\} \\ -1 & \min\{D_n - 1, L_n\} < v \le \min\{D_n, L_n\} \end{cases} \tag{15}$$

Then

$$\begin{aligned} \lim_{\tau\to\infty} \sum_{l=1}^{NT(\tau)} r_t^e &= \lim_{\tau\to\infty} \sum_{l=1}^{NT(\tau)} \sum_{n=1}^{N(\tau)} \int_{\tau_{t-1}}^{\tau_t} w_n p_n \delta_n(v)\, \mathrm{d}v \\ &= \lim_{\tau\to\infty} \sum_{n=1}^{N(\tau)} \int_{R_n}^{\min\{D_n, L_n\}} w_n p_n \delta_n(v)\, \mathrm{d}v \end{aligned} \tag{16}$$

Assume that

$$\begin{aligned} SH1 &= \{n | L_n = D_n, n \le N(\tau)\} \\ SH2 &= \{n | L_n < D_n, n \le N(\tau)\} \end{aligned} \tag{17}$$

and

$$\begin{aligned} &\lim_{\tau\to\infty} \sum_{t=1}^{NT(\tau)} r_t^e \\ &= \lim_{\tau\to\infty} \left(\sum_{n\in SH1} \left[\int_{R_n}^{D_n - 1} w_n p_n \delta_n(v)\, \mathrm{d}v \right. \right. \\ &\quad \left. + \int_{D_n - 1}^{D_n} w_n p_n \delta_n(v)\, \mathrm{d}v \right] \end{aligned}$$

$$+\sum_{n\in SH2}\int_{R_n}^{L_n} w_n p_n \delta_n(v)\,\mathrm{d}v\Bigg)$$
$$=\lim_{\tau\to\infty}\left(\sum_{n\in SH1}\left[0+\left(-w_n p_n\right)\right]+0\right)$$
$$=\lim_{\tau\to\infty}\sum_{n\in SH1}-w_n p_n \tag{18}$$

Meanwhile,

$$\lim_{\tau\to\infty}\text{object expired}=\lim_{\tau\to\infty}\sum_{n=1}^{N(\tau)}\varphi_n p_n w_n$$
$$=\lim_{\tau\to\infty}\sum_{n\in SH1} p_n w_n \tag{19}$$

Therefore

$$\min\lim_{\tau\to\infty}\text{object expired}\Leftrightarrow\max\lim_{\tau\to\infty}\sum_{t=1}^{NT(\tau)} r_t^e \tag{20}$$

Since $\delta_n(v)$ is a function defined within $[E_n, D_n]$, we have

$$r_t^e=\sum_{n=1}^{N(\tau)}\int_{\tau_{t-1}}^{\tau_t} w_n p_n \delta_n(v)\,\mathrm{d}v$$
$$=\sum_{n\in SY(\tau_{t-1},\tau_t)}\int_{\tau_{t-1}}^{\tau_t} w_n p_n \delta_n(v)\,\mathrm{d}v \tag{21}$$

In general, we have

$$\min\lim_{\tau\to\infty}\text{object}=\min\lim_{\tau\to\infty}(\text{object expired}$$
$$+\text{object tardiness})\Leftrightarrow\max\lim_{\tau\to\infty}\sum_{t=1}^{NT(\tau)}\left(r_t^e+r_t^d\right) \tag{22}$$

and the reward function can be expressed as

$$r_t=r_t^d+r_t^e$$
$$=\sum_{n\in SY(\tau_{t-1},\tau_t)} w_n\int_{\tau_{t-1}}^{\tau_t}\left(\sigma_n(v)+p_n\delta_n(v)\right)\mathrm{d}v \tag{23}$$

3.4. Settings of workshop simulation environment

There are two types of parameters in our environment. The parameters of the workshop are set by us and remain fixed during the training process for a certain workshop environment. Table 1 shows the parameters of the workshop production resources. And the parameters of each job is subject to a certain random distribution, as is shown in Table 2. In the environment, there can only be one new job for each time step. λ is the time interval between the last job's arrival and the next job's arrival. p is the time it will take to process a certain job. w is the priority of one job. u is the buffer size the job will use once it is completed but not delivered. k is the urgent coefficient we use to define a job's delivery time and deadline, as is shown in (24) and (25).

Table 1. Workshop production resource parameters.

Name	Symbol	Description
Number of machines	M	The number of machines in the simulation environment
Number of tools	I	The number of tools in the simulation environment
Number of workers	J	The number of workers in t he simulation environment
Length of job queue	K	When the queue is full with K jobs, new jobs will be rejected
Size of buffer	U	Capacity of the buffer that stores complete jobs
Maximum jobs	N	Maximum number of jobs generated in one episode

Table 2. Configurable distribution parameters for job generating.

Name	Symbol	Distribution
Time interval between arrivals	λ	Uniform integer distribution in range $[\lambda_{\min}, \lambda_{\max})$
Process time of jobs	p	Uniform integer distribution in range $[p_{\min}, p_{\max})$
Priority of jobs	w	Uniform integer distribution in range $[w_{\min}, w_{\max})$
Buffer use of jobs	u	Uniform integer distribution in range $[u_{\min}, u_{\max})$
Urgent coefficient of jobs	k	Uniform distribution in range $[k_{\min}, k_{\max})$

$$E=\text{floor}(R+p\times k) \tag{24}$$

R in (24) is the release time of the job, which means the time it arrives in our environment. And floor in (24) is the function of rounding down. The deadline is set as (25) for simplification.

$$D=E+p \tag{25}$$

4. The reinforcement learning algorithm for workshop scheduling problem

4.1. Algorithm

In this paper, we use PPO algorithm shown below (Schulman et al. 2017) to solve the problem of workshop scheduling's model-free MDP. PPO has a good stability during training, and it has been used in multi-action output problems (Berner et al. 2019).

The generalised advantage estimation $\hat{A}_t$ shown below is used to replace long-term returns

$$\hat{A}_t=\delta_t+(\gamma\lambda_l)\delta_{t+1}+\cdots+\cdots+(\gamma\lambda_l)^{T-t+1}\delta_{T-1} \tag{26}$$

Algorithm 1 Proximal policy optimization

for iteration=1, 2,.. . **do**
  for parallel environment=1, 2, . . . , N_{par} **do**
    Run policy $\pi_{\theta old}$ in environment for T steps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L_{sur} wrt θ, with K_{epo} epochs and minibatch size $M_{bat} \leq N_{par}T$
  $\theta_{old} \leftarrow \theta$
end for

where

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{27}$$

and $V(s_t)$ is the state value function; r_t is the reward.

The surrogate L_{sur} shown below is the objective function of PPO algorithm

$$L_t^{CLIP+VF+S}(\theta) = \widehat{E}_t\left[L_t^{CLIP} - c_1 L_t^{VF} + c_2 S\left[\pi_\theta\right](s_t)\right] \tag{28}$$

where

$$\begin{aligned} L_t^{CLIP} &= \hat{E}_t\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}\left(r_t(\theta), 1-\varepsilon, 1+\varepsilon\right)\hat{A}_t\right)\right] \\ L_t^{VF} &= \left(V_\theta(s_t) - V_t^{\text{targ}}\right)^2 \\ r_t(\theta) &= \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \end{aligned} \tag{29}$$

In (28) and (29), $V_\theta(s_t)$ is the state value function, V_t^{targ} is the target value function, and $S[\pi_\theta](s_t)$ is the entropy bonus.

4.2. Multi-modal scheduling neural network

PPO is a stability-enhanced RL algorithm based on actor-critic (AC) structure for neural network training. An AC algorithm can be divided into a policy network fitting output acts and a state value network fitting state value function. While in this paper, two networks are integrated into a unified network with multi-head output through a hard sharing technique (Silver et al. 2017). The multi-modal scheduling neural network can be divided into feature extraction, deep fusion, and output layer. The network used in our paper is shown in Figure 4.

Since the workshop state is represented as a multi-modal feature with both graphical and numerical information, in the feature extraction stage, a multi-layer convolutional network similar to nature-CNN is used to extract deeper features from graphical state features, and a multi-layer fully connected (FC) network is used to extract deeper features from numerical information. The above process can be regarded as encoding stage.

In the deep fusion stage, two feature vectors are spliced. Then, a FC layer of 1024 hidden units and a rectified linear unit (RELU) are used to integrate the spliced features. For every output head, a separate FC layer is used to learn knowledge about the corresponding scheduling task.

The output network can still be studied separately as a policy network and a value function network. For the policy network, the resources of job, machine, tool, and worker are scheduled simultaneously in a single vector through a multi-label technique. Specifically, in the policy network, each output head is obtained through one FC layer going through the Softmax activation function. When being activated by Softmax function, the corresponding action masks are used to normalise the probability of available actions. Since every output head corresponds to one kind of resources to be scheduled, the number of neurons in each output head and FC layer is equal to the total number of corresponding resources plus one. The extra action means no resource will be selected. For example, if the length of job queue is $K = 2$ and the number of machines is $M = 3$, the number of neurons in the policy output head of job and machine is 3 and 4 respectively. If the policy output head of job outputs $[0, 1, 0]^T$, the scheduling system will select the job with the position index of 2, assuming the index starts with 1. However, if the machine head outputs $[0, 0, 0, 1]^T$, which means no machine will be selected, the scheduling decision becomes invalid. This part can be seen as the decoding stage of the model.

4.3. Multi-label output policy

Usually, the basic PPO algorithm chooses one output act in a discrete action space for every decision timestep. For a more complex problem, such as Super Mario Bro game (Kauten 2018), combinational acts (jump and direction control) can be encoded as one selectable single act in its action space. But for our problem, the scheduling of job, machine, tool, and worker is too complex to realise with combinational acts.

To solve this problem, we need to decouple the scheduling of different resources. First, we use independent acts to schedule different kinds of resources. Second, acts for every kind of resources are generated on different label nodes of a multi-label policy network. Before using the multi-label policy network, the objective function and policy entropy of PPO without clipping are

$$\max_\theta \hat{\mathbb{E}}_{s,a\sim\pi_{\theta_{\text{old}}}}\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}\hat{A}_t\right] \tag{30}$$

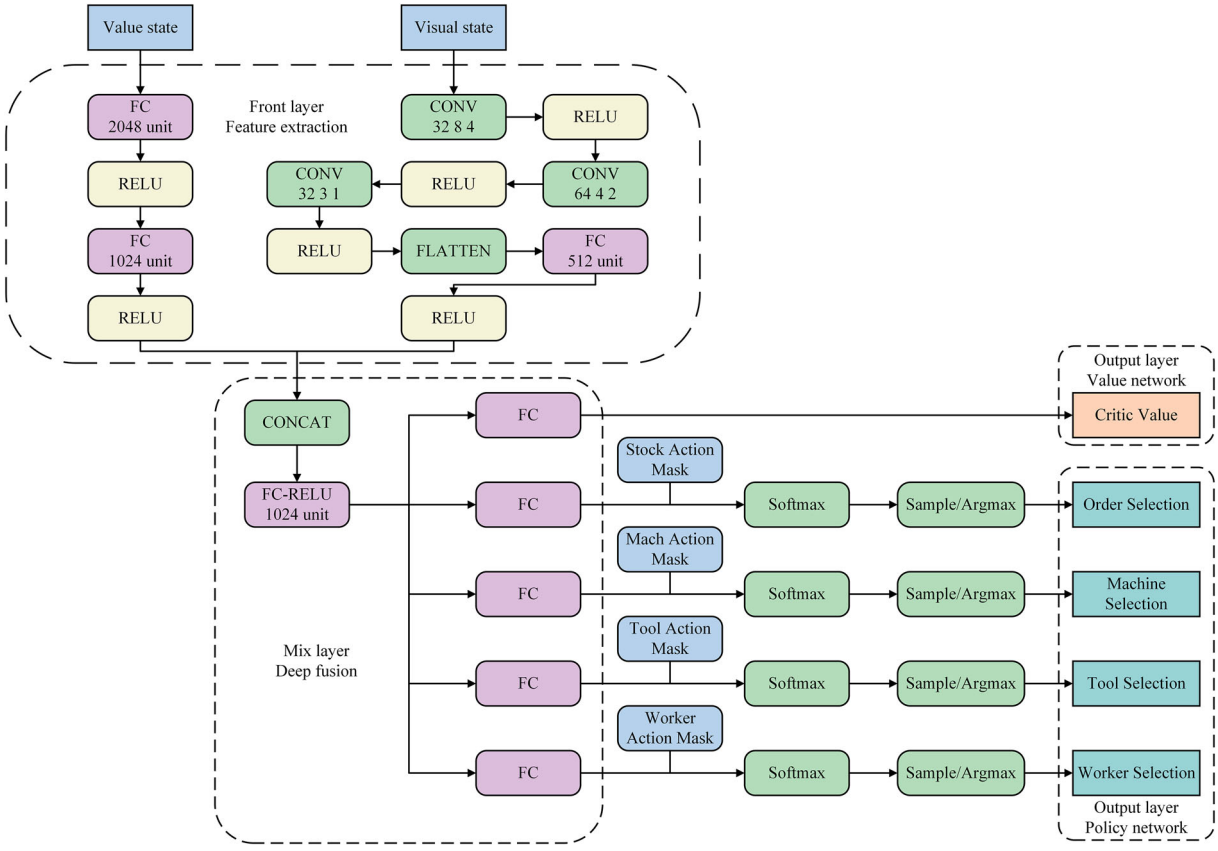

Figure 4. The dual-path hybrid neural network.

$$H = S[\pi_\theta](s_t) \tag{31}$$

Assuming that every act is $a = (a^0, \ldots, a^{N_a-1})$, after introducing multi-label policy network, the objective function and policy entropy become

$$\max_\theta \hat{\mathbb{E}}_{s,a\sim\pi_{\theta_{old}}} \left[\prod_{i=0}^{N_a-1} \frac{\pi_\theta\left(a_t^{(i)} \mid s_t\right)}{\pi_{\theta_{\text{old}}}\left(a_t^{(i)} \mid s_t\right)} \hat{A}_t \right] \tag{32}$$

$$H = \sum_{i=0}^{N_a-1} S^{(i)}[\pi_\theta](s_t) \tag{33}$$

where $S^{(i)}$ is the entropy of each act. Through multi-label policy network, the scheduling of resources is decoupled moderately, enabling our model to explore and evaluate the scheduling environment with more diversified act combinations.

4.4. The realisation of action mask

To improve the training efficiency and to reduce useless exploration, the technique of action mask is introduced, which is a Boolean vector. In action mask, 0 means the act is suppressed, and 1 means the act is available. The length of action mask is equal to the size of the constrained action space. The realisation of action mask is shown below:

$$logit_masked = logit + log(action_mask + e) \tag{34}$$

where *logit* is an unnormalised value of an act probability generated by policy network, and *e* is a extremely small constant that can avoid overflow. *logit_masked* is the *logit* processed by the action mask layer. After *logit_masked* goes through Softmax layer and gets normalised, the probabilities of acts suppressed by action masks will be extremely close to 0. Thus, the action output by sample or argmax will not choose these unavailable acts. Essentially, action mask is a pruning method using prior knowledge that can encode the information about constraints into the policy network, realising a more efficient exploration and training for our scheduling model.

4.5. Custom decision interval

Custom decision interval or frame skip means the model only makes decisions on keyframes rather than every frame. Compared to normal time-sequential frame skip, we add an interruption mechanism to force the system

return the current state to the model and run a decision process, which can be understood as on-demand decision. This frame skip mechanism can accelerate the training process because it decreases the possibility that our model makes meaningless decisions and learns from them. The maximum decision interval is set to 6 in our experiments.

5. Experiments and results

Our scheduling environment is built on the basis of OpenAI's Gym (Brockman et al. 2016), and the model is realised using Pytorch (Paszke et al. 2019). All experiments are run on a single CPU Intel Xeon E3 1230V2 @3.7GHz and two Nvidia GEFORCE GTX 1060 6G GPUs. To verify our model's performance in dynamic scheduling environments, we compare our model to heuristic methods under environments with different load.

5.1. Hyperparameter verification

Since there are lots of adjustable hyperparameters in our scheduling model, to control the amount of sensitivity experiments, the hyperparameters of PPO are preset to values performing good in classic RL tasks, and only the hyperparameters of network training are adjustable in experiments. The value range of hyperparameters is shown in Table 3. Taguchi's method is used to conduct the experiments about parameters in Table 4, and results are used for comparison with the experience based base group of parameters $\{2.5 \times 10^{-4}, 4, 256\}$. All results are the scaled penalty averaged over 96 examples. Scaled penalty means the penalty of each instance (episode) scaled by a factor of $1/N$, where N is maximum number of jobs and is set to 600. All models are trained for 8.5-10.5 hours, and the workshop environment parameters are set to $\{\lambda_{\max} = 10, M = 5, I = 7, J = 6, K = 15, Z = 10\}$.

From Table 4, it can be seen that the parameters of the base group in the table have excellent performance compared with the parameters of other groups, indicating the effectiveness of the parameter setting of base group. At the same time, it can be seen that the results of the group with larger learning rate parameters are significantly worse than those of the other groups, indicating that the performance of the algorithm is more sensitive to learning rate. Lastly, the parameter selection of the scheduling model is detailed in Table 5.

Table 3. Hyperparameters discrete partition.

Name	Partition
Learning rate	$\{2.5 \times 10^{-2}, 2.5 \times 10^{-3}, 2.5 \times 10^{-4}\}$
Batch size	$\{128, 256, 512\}$
Epoch	$\{2, 4, 8\}$
Gradient clipping	$\{0.20, 0.50, 0.80\}$

Table 4. Sensitivity analysis with Taguchi method.

No.	Learning rate	Epoch	Batch size	Gradient clipping	Result
1	2.5×10^{-2}	2	128	0.20	51.02
2	2.5×10^{-2}	4	256	0.50	50.74
3	2.5×10^{-2}	8	512	0.80	50.99
4	2.5×10^{-3}	2	256	0.80	37.53
5	2.5×10^{-3}	4	512	0.20	39.67
6	2.5×10^{-3}	8	128	0.50	44.79
7	2.5×10^{-4}	2	512	0.50	36.62
8	2.5×10^{-4}	4	128	0.80	36.22
9	2.5×10^{-4}	8	256	0.20	36.78
base	2.5×10^{-4}	4	256	0.50	36.09

Table 5. Algorithm hyperparameters.

Name	Symbol	Value
Learning rate	α	2.5×10^{-4}
Discount factor	γ	0.99
Clipping coefficient	ε	0.10
Number of parallel environments	N_{par}	8
Steps	T	1024
Batch size	M_{bat}	256
Loss coefficient of value function	c_1	0.50
Entropy coefficient	c_2	0.01
Epoch	K_{epo}	4
Generalised advantage estimation coefficient	λ_l	0.95
Gradient clipping	β	0.50

5.2. Performance comparison

Nineteen experiments of different production resource and job distribution are done in this paper. Each experiment includes a test set of 240 instances. A PPO model is trained separately for each experiment and its performance is compared with four heuristic rules: WSPT, WMDD, ATC, WCOVERT and two reinforcement methods: DeepRM (Mao et al. 2016), DeepRM2 (Ye et al. 2018). DeepRM and DeepRM2 adopt the same setting as PPO in state feature, reward function, state transition and training environment. As for the action space, the DeepRM method only outputs the job scheduling actions and does not use the action mask technique. The machine, tool, and worker scheduling actions are generated by the least flexible (LF) rule (Lamothe et al. 2012). In addition, DeepRM2 adopts two training stages: imitation learning and policy gradient, and the rule used for imitation learning is WCOVERT. The DeepRM2 model that only uses imitation training is called DMR2_WCO, and the two-stage trained model is called DRM2.

Figure 5 shows the training process of the model in five experiments, where the x axis represents the number of iterations of the PPO algorithm, and the y axis represents the scaled penalty averaged over 100 instances (scaled factor is $1/N$, $N = 600$). It can be observed from Figure

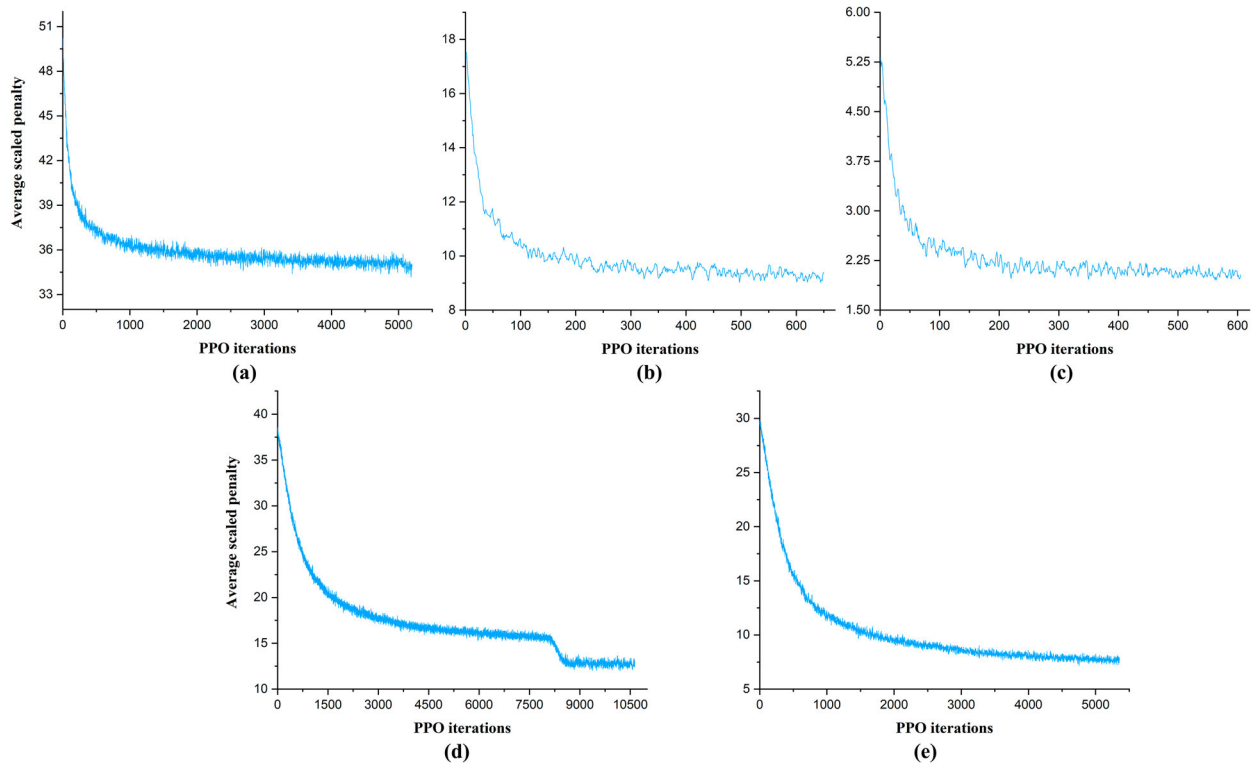

Figure 5. The training process curves with different workshop parameters: (a) $M = 5, \lambda_{\max} = 10$; (b) $M = 10, \lambda_{\max} = 15$; (c) $M = 15, \lambda_{\max} = 10$; (d) $M = 20, \lambda_{\max} = 3$; (e) $M = 25, \lambda_{\max} = 3$.

5 that in the earlier phase, our model has a larger exploration rate and its scheduling performance is improved rapidly, which is known as pre-training. In the later phase, the pattern of the model will gradually be transformed into exploitation from exploration, which is known as fine tuning.

The experiment can be divided into 5 groups according to the number of production resources. In each group of experiment, the number of production resources (M, I, J, K, U) remains the same and different $\lambda_{\max}$ are used to observe the adaptability of our model under different production stress. The average scaled penalty in test set of PPO and other rules are illustrated in Table 6.

It can be observed that in most experiments, WCOVERT performs better than other rules, and WSPT performs worst among these rules. It can be observed that the scheduling performance of DeepRM is much worse than WSPT, which indicates that DeepRM trained through REINFORCE algorithm may not be suitable to this environment. DeepRM2 using an imitation learning method achieves a scheduling performance slightly worse than WCOVERT. Because DeepRM2 imitates the policy of WCOVERT rule, it will be very hard for the the DeepRM2 model to achieve a better performance than WCOVERT. Meanwhile, the two-stage trained DRM2 model performs worse than DRM2_WCO, and the reason is likely to be that PG method is too sensitive to the hyperparameters, which is also a problem for DeepRM in this environment. Nevertheless, the PPO model succeeds to exceed all methods in performance. In conclusion, the PPO model can handle the production resource scheduling policy with the object of minimising the global processing penalty, through successful exploration and evaluation. From *gap* in Table 6, it can also be observed that the scheduling performance of PPO is at least 1.138% better than other algorithms, and when compared to the best of the rest, the performance of PPO is at most 30.196% better.

5.3. Result analysis

In each group of experiments, it can be observed that as $\lambda_{\max}$ becomes smaller, the performance difference between rules becomes larger. For example, in the third group of experiments, when $\lambda_{\max} = 10$, the difference of the average scaled penalty between WCOVERT and ATC is 0.084, however, when $\lambda_{\max} = 4$, the difference increases to 3.729. That is because when $\lambda_{\max}$ decreases, the busy periods become longer and the idle periods become shorter. Under a stressful environment, irrational scheduling policy will cause problems such as low usage of resources, backlog of jobs with high priority, and queue blocking. As a result, we will see larger performance difference.

Table 6. Results of average scaled penalty in test set for different methods.

	heuristic rules				reinforcement methods				
λ_{max}	WSPT	WMDD	ATC	WCOVERT	DeepRM	DRM2_WCO	DRM2	PPO	gap(%)
				Group 1: M = 5, I = 7, J = 6, K = 15, U = 10					
10	44.688	37.524	37.536	36.540	69.946	36.776	68.444	34.568	5.704
20	22.075	17.306	16.821	17.234	81.709	17.1988	73.242	15.964	5.368
30	10.877	8.850	8.604	8.902	93.294	8.732	58.538	8.423	2.149
40	4.613	4.001	3.909	4.022	70.651	4.033	40.420	3.865	1.138
				Group 2: M = 10, I = 12, J = 13, K = 30, U = 15					
10	29.304	20.958	20.695	19.562	63.388	19.220	74.432	18.206	5.571
15	15.005	10.186	9.651	9.726	51.975	9.724	62.090	8.831	9.285
20	7.465	5.424	5.155	5.255	53.146	5.242	95.770	4.930	4.563
25	3.854	3.007	2.921	2.979	52.045	2.897	75.512	2.777	4.348
				Group 3: M = 15, I = 17, J = 20, K = 45, U = 30					
4	34.336	22.034	22.310	18.581	64.456	18.784	65.076	17.830	4.212
6	19.664	10.638	10.070	8.376	67.221	8.132	71.426	7.405	9.824
8	9.538	4.941	4.396	4.062	89.611	3.975	35.502	3.463	14.810
10	4.143	2.351	2.135	2.051	82.354	1.992	92.330	1.915	4.030
				Group 4: M = 20, I = 23, J = 18, K = 60, U = 40					
3	32.865	19.354	19.746	15.147	75.444	15.418	60.934	11.634$^{(1)}$	30.196
4	23.053	11.831	11.410	8.278	62.408	8.153	77.773	8.017	1.706
5	14.763	6.750	5.974	4.521	58.044	4.614	78.031	3.921	15.302
6	8.608	3.882	3.269	2.725	70.987	2.660	70.500	2.354$^{(2)}$	13.031
				Group 5: M = 25, I = 20, J = 25, K = 70, U = 50					
3	24.451	11.914	11.506	7.473	60.479	8.536	55.737	6.718	11.238
4	14.219	5.972	5.149	3.301	92.654	3.501	60.281	3.124	5.665
5	6.841	2.732	2.150	1.687	58.194	1.691	63.168	1.490	13.221

Some parameters of workshop environment are fixed for different experiments: $\lambda_{min} = 1$, $p \in [5, 40)$, $w, k, u \in [1, 5)$, $N = 600$; $h_{ATC} = 1$, and $K_{WCO} = 1$. Define $gap = \frac{Y_{best} - Y_{PPO}}{Y_{PPO}} \times 100\%$, where Y_{best} is the best performance in this experiment except for PPO, and Y_{PPO} is the performance of PPO.

In Table 7, the training time of each network of our model are illustrated. For most of the networks, we stop the training once it began to perform better than WCOVERT, which means their performance still has space for further improvement. In group 4, the training does not stop early: the network is trained for 62 hours in total, as a result, the performance difference between this network and WCOVERT reaches 3.513, which is larger than other experiments. The average running time of our model and other rules for each experiment are also shown in Table 7. Considering the running time of different rules are very close, we use one average to represent all rules. Though the PPO model consumes more performance, its running time is not greater than three times the consumption of rules, therefore, we think it is practical to replace scheduling rules with our model.

Table 7. Results of running time of PPO models and heuristic rules.

λ_{max}	PPO training time(h)	PPO test time(s)	heuristic rule test time(s)
		$M = 5, I = 7, J = 6, K = 15, U = 10$	
10	22	4.927	1.866
20	13	9.036	3.512
30	10	12.913	5.024
40	16	13.637	5.222
		$M = 10, I = 12, J = 13, K = 30, U = 15$	
10	18	5.160	2.003
15	11	7.424	2.919
20	9	9.411	3.704
25	(lost)	11.611	4.524
		$M = 15, I = 17, J = 20, K = 45, U = 30$	
4	26	2.830	1.079
6	19	3.956	1.475
8	20	5.014	1.812
10	10	5.852	2.176
		$M = 20, I = 23, J = 18, K = 60, U = 40$	
3	62	2.534	0.965
4	24	3.169	1.172
5	18	3.725	1.406
6	16	4.281	1.624
		$M = 25, I = 20, J = 25, K = 70, U = 50$	
3	38	2.800	1.024
4	16	3.488	1.285
5	16	4.106	1.492

The tolerance of training time is ±1 hour. Test time is the average test time of one instance.

Last but not least, an extra experiment about the transfer learning performance of our model is also done. In group 4, we exchange the simulation environments of model trained when $\lambda_{max} = 6$ and $\lambda_{max} = 3$. It can be observed from the results in Table 8 that the model trained in a relatively stressful environment can handle a less stressful environment, only slightly worse than the model trained in such environment. However, we can also find that it is difficult for the policy acquired in a less stressful environment to adapt to an environment with higher load.

Table 8. Experimental results of the transfer learning performance (average scaled penalty).

λ_{max}	WSPT	WMDD	ATC	WCOVERT	PPO$^{(1)}$	PPO$^{(2)}$
3	32.865	19.354	19.746	15.147	11.634	39.530
6	8.608	3.882	3.269	2.725	2.410	2.354

PPO$^{(1),(2)}$ correspond to the models marked with$^{(1)}$ and$^{(2)}$ in Table 6.

6. Conclusions

In this paper, RL method is used to solve the multi-resource constrained dynamic workshop scheduling problem. Considering the dynamic arrival of jobs, the scheduling optimisation problem is transformed into an MDP and solved by the PPO algorithm. The multi-modal hybrid neural network is trained by PPO, and scheduling acts are generated according to the numerical and graphical features representing the workshop state. In order to solve the problem, the multi-label technique is used to decouple different production resources, and action masks are used to prune invalid acts based on prior constraints. The experimental results show that the scheduling model can improve the scheduling performance through training and will converge gradually. The proposed method can obtain better scheduling policy compared with other existing methods. In the future, we plan to introduce the constraint of process order into the scheduling problem to further enhance the application in the actual production.

Disclosure statement

No potential conflict of interest was reported by the authors.

Funding

The research is supported by the National Key Research and Development Program of China (Grant No. 2017YFE0101400).

Notes on contributors

Peng Cheng Luo received the B.S. degree in mechanical engineering and automation from the East China University of Science and Technology, Shanghai, China, in 2018, the M.S. degree in mechanical engineering from Tongji University, Shanghai, China, in 2021. His research interests include modelling and analysing of manufacturing systems with machine learning technics and their application to production scheduling.

Huan Qian Xiong received the B.S. degree in electronic science and technology from Tongji University, Shanghai, China, in 2018, the M.E. degree in integrated circuit engineering from Tongji University, Shanghai, China, in 2021. His research interests include antenna design, application of metasurfaces, and machine learning in multi-agent system.

Bo Wen Zhang received the B.S. degree in mechanical design, manufacturing and automation from Tongji University, Shanghai, China, in 2018, the M.E. degree in mechanical engineering from Tongji University, Shanghai, China, in 2021. His research interests include preventive maintenance and production scheduling of manufacturing systems.

Jie Yang Peng received the B.S. degree in mechanical engineering from the East China University of Science and Technology, Shanghai, China, in 2013, the M.S. degree in mechanical engineering from Tongji University, Shanghai, China, in 2017. Since 2017, he has been working towards Ph.D. degree in Tongji University. His research interests include modelling and analysing of manufacturing facilities with machine learning technics and their application to preventive maintenance.

Zhao Feng Xiong received the B.S. degree in Electrical Engineering and its Automation from the Huazhong University of Science and Technology (Wenhua College), Wuhan, Hubei, China, in 2018. His research interests include distributed management and analysis of industrial IoT using cloud-native technologies.

ORCID

Zhao Feng Xiong http://orcid.org/0000-0002-4291-1692

References

Aydin, M. E., and E. Öztemel. 2000. "Dynamic Job-shop Scheduling Using Reinforcement Learning Agents." *Robotics and Autonomous Systems* 33 (2-3): 169–178.

Bellemare, M. G., W. Dabney, and R. Munos. 2017. "A Distributional Perspective on Reinforcement Learning." In *Paper presented at 34th International Conference on Machine Learning*, Sydney, August 6–11.

Berner, C., G. Brockman, B. Chan, V. Cheung, P. Dębiak, C. Dennison, D. Farhi, et al. 2019. "Dota 2 With Large Scale Deep Reinforcement Learning." *arXiv preprint arXiv:1912.06680*.

Brockman, G., V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. 2016. "Openai Gym." *arXiv preprint arXiv:1606.01540*.

Cao, ZhengCai, ChengRan Lin, and MengChu Zhou. 2019. "A Knowledge-based Cuckoo Search Algorithm to Schedule a Flexible Job Shop with Sequencing Flexibility." *IEEE Transactions on Automation Science and Engineering* 18 (1): 56–69.

Cao, ZhengCai, ChengRan Lin, MengChu Zhou, and Ran Huang. 2018. "Scheduling Semiconductor Testing Facility by Using Cuckoo Search Algorithm with Reinforcement Learning and Surrogate Modeling." *IEEE Transactions on Automation Science and Engineering* 16 (2): 825–837.

Cao, ZhengCai, ChengRan Lin, MengChu Zhou, and JiaQi Zhang. 2020. "Surrogate-assisted Symbiotic Organisms Search Algorithm for Parallel Batch Processor Scheduling." *IEEE/ASME Transactions on Mechatronics* 25 (5): 2155–2166.

Carrol, D. 1965. "Heuristic Sequencing of Jobs With Single and Multiple Components." PhD diss., Sloan School of Management, Massachusetts Institute of Technology.

Dou, Jianping, Jun Li, Dan Xia, and Xia Zhao. 2020. "A Multi-objective Particle Swarm Optimisation for Integrated Configuration Design and Scheduling in Reconfigurable Manufacturing System." *International Journal of Production Research* 59 (5): 1–21.

Fu, Yaping, MengChu Zhou, Xiwang Guo, and Liang Qi. 2019. "Scheduling Dual-objective Stochastic Hybrid Flow Shop with Deteriorating Jobs Via Bi-population Evolutionary Algorithm." *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 50 (12): 5037–5048.

Gao, Kaizhou, Zhiguang Cao, Le Zhang, Zhenghua Chen, Yuyan Han, and Quanke Pan. 2019. "A Review on Swarm Intelligence and Evolutionary Algorithms for Solving Flexible Job Shop Scheduling Problems." *IEEE/CAA Journal of Automatica Sinica* 6 (4): 904–916.

Hessel, M., J. Modayil, H. Van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver. 2018. "Rainbow: Combining Improvements in Deep Reinforcement Learning." In *Paper presented at 32nd AAAI Conference on Artificial Intelligence*, New Orleans, February 02–07.

Kauten, Christian. 2018. "Super Mario Bros for OpenAI Gym." GitHub. https://github.com/Kautenja/gym-super-mario-bros.

Komaki, G. M., Shaya Sheikh, and Behnam Malakooti. 2019. "Flow Shop Scheduling Problems with Assembly Operations: a Review and New Trends." *International Journal of Production Research* 57 (10): 2926–2955.

Lamothe, Jacques, François Marmier, Matthieu Dupuy, Paul Gaborit, and Lionel Dupont. 2012. "Scheduling Rules to Minimize Total Tardiness in a Parallel Machine Problem with Setup and Calendar Constraints." *Computers & Operations Research* 39 (6): 1236–1244.

Li, Yingli, Xinyu Li, Liang Gao, Biao Zhang, Quan-Ke Pan, M. Fatih Tasgetiren, and Leilei Meng. 2021. "A Discrete Artificial Bee Colony Algorithm for Distributed Hybrid Flowshop Scheduling Problem with Sequence-dependent Setup Times." *International Journal of Production Research* 59 (13): 3880–3899.

Mahadevan, Sridhar. 1996. "Average Reward Reinforcement Learning: Foundations, Algorithms, and Empirical Results." *Machine Learning* 22 (1-3): 159–195.

Mao, Hongzi, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. 2016. "Resource Management With Deep Reinforcement Learning." In *Paper presented at Proceedings of the 15th ACM Workshop on Hot Topics in Networks*, Atlanta, November 9–10.

Mnih, Volodymyr, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. "Asynchronous Methods for Deep Reinforcement Learning." In *Paper Presented at International Conference on Machine Learning*, New York City, June 19–24.

Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. "Playing Atari With Deep Reinforcement Learning." In *Paper presented at Neural Information Processing Systems*, Lake Tahoe, December 5–10.

Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, and Alex Graves, et al. 2015. "Human-level Control Through Deep Reinforcement Learning." *Nature* 518 (7540): 529–533.

Oukil, Amar, and Ahmed El-Bouri. 2021. "Ranking Dispatching Rules in Multi-objective Dynamic Flow Shop Scheduling: A Multi-faceted Perspective." *International Journal of Production Research* 59 (2): 388–411.

Park, Junyoung, Jaehyeong Chun, Sang Hun Kim, Youngkook Kim, and Jinkyoo Park. 2021. "Learning to Schedule Job-shop Problems: Representation and Policy Learning Using Graph Neural Network and Reinforcement Learning." *International Journal of Production Research* 59 (11): 3360–3377.

Paszke, Adam, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, and Trevor Killeen, et al. 2019. "Pytorch: An Imperative Style, High-Performance Deep Learning Library." In *Advances in Neural Information Processing Systems*, 8026–8037.

Schaul, Tom, John Quan, Ioannis Antonoglou, and David Silver. 2015. "Prioritized Experience Replay." In *Paper presented at 4th International Conference on Learning Representations*, San Juan, May 2–4.

Schulman, John, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. 2015. "Trust Region Policy Optimization." In *Paper Presented at International Conference on Machine Learning*, Lille, July 6–15.

Schulman, John, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. 2015. "High-Dimensional Continuous Control Using Generalized Advantage Estimation." In *Paper Presented at 4th International Conference on Learning Representations*, San Juan, May 2–4.

Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. "Proximal Policy Optimization Algorithms." *arXiv preprint arXiv:1707.06347*.

Silver, David, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, and Thomas Hubert, et al. 2017. "Mastering the Game of Go Without Human Knowledge." *Nature* 550 (7676): 354–359.

Sutton, Richard S., David A. McAllester, Satinder P. Singh, and Yishay Mansour. 2000. "Policy Gradient Methods for Reinforcement Learning With Function Approximation." In *Advances in Neural Information Processing Systems*, 1057–1063.

Touzout, Faycal A., and Lyes Benyoucef. 2019. "Multi-objective Multi-unit Process Plan Generation in a Reconfigurable Manufacturing Environment: a Comparative Study of Three Hybrid Metaheuristics." *International Journal of Production Research* 57 (24): 7520–7535.

Van Hasselt, Hado, Arthur Guez, and David Silver. 2016. "Deep Reinforcement Learning With Double Q-Learning." In *Paper Presented at 30th AAAI Conference on Artificial Intelligence*, Phoenix, February 12–17.

Vepsalainen, Ari P. J., and Thomas E. Morton. 1987. "Priority Rules for Job Shops with Weighted Tardiness Costs." *Management Science* 33 (8): 1035–1047.

Volgenant, A., and E. Teerhuis. 1999. "Improved Heuristics for the N-job Single-machine Weighted Tardiness Problem." *Computers & Operations Research* 26 (1): 35–44.

Wang, Jufeng, Chunfeng Liu, and MengChu Zhou. 2020. "Improved Bacterial Foraging Algorithm for Cell Formation and Product Scheduling Considering Learning and Forgetting Factors in Cellular Manufacturing Systems." *IEEE Systems Journal* 14 (2): 3047–3056.

Wang, Ling, and Jiawen Lu. 2019. "A Memetic Algorithm with Competition for the Capacitated Green Vehicle Routing Problem." *IEEE/CAA Journal of Automatica Sinica* 6 (2): 516–526.

Wang, Ziyu, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas. 2016. "Dueling Network Architectures for Deep Reinforcement Learning." In *Paper Presented at International Conference on Machine Learning*, New York City, June 19–24.

Wang, Yi-Chi, and John M. Usher. 2005. "Application of Reinforcement Learning for Agent-based Production Scheduling." *Engineering Applications of Artificial Intelligence* 18 (1): 73–82.

Watkins, Christopher J. C. H., and Peter Dayan. 1992. "Q-learning." *Machine Learning* 8 (3-4): 279–292.

Wei, Yingzi, and Mingyang Zhao. 2004. "Composite Rules Selection Using Reinforcement Learning for Dynamic Job-Shop Scheduling." In *Paper presented at IEEE Conference on Robotics, Automation and Mechatronics*, Singapore, Dec 1–3.

Yang, Shengluo, and Zhigang Xu. 2021. "The Distributed Assembly Permutation Flowshop Scheduling Problem with Flexible Assembly and Batch Delivery." *International Journal of Production Research* 59 (13): 4053–4071.

Ye, Yufei, Xiaoqin Ren, Jin Wang, Lingxiao Xu, Wenxia Guo, Wenqiang Huang, and Wenhong Tian. 2018. "A New Approach for Resource Scheduling With Deep Reinforcement Learning." *arXiv preprint arXiv:1806.08122*.

Zhang, Zhicong, Li Zheng, Na Li, Weiping Wang, Shouyan Zhong, and Kaishun Hu. 2012. "Minimizing Mean Weighted Tardiness in Unrelated Parallel Machine Scheduling with Reinforcement Learning." *Computers & Operations Research* 39 (7): 1315–1324.

Zhang, Zhicong, Li Zheng, and Michael X. Weng. 2007. "Dynamic Parallel Machine Scheduling with Mean Weighted Tardiness Objective by Q-Learning." *The International Journal of Advanced Manufacturing Technology* 34 (9–10): 968–980.

Appendix. Mathematical model

In order to accurately describe the dynamic workshop scheduling problem with multi-resource constraints, the linear programming model is adopted to define the problem studied in this paper. The definitions of relevant variables and symbols in the linear programming model of the problem in this paper are shown in Table A9.

The linear programming model of dynamic workshop scheduling problem with multi-resource constraints is defined as follows:

$$\textit{Minimize}\ \text{object} = \sum_{n=1}^{N} w_n[(L_n - E_n)^+ + \varphi_n p_n] \tag{A1}$$

Table A9. Related variables and symbol definitions.

Symbol	Description
	Notations
M	The number of machines in the workshop
I	The number of tools in the workshop
J	The number of workers in the workshop
K	Maximum length of queue waiting for processing
U	Capacity of the buffer that stores complete jobs
N	The total number of arrival jobs
τ	The current moment
	Indices
n, n'	Indices of jobs
m	Index of machine
i	Index of tool
j	Index of worker
	Basic parameters
p_n	Process time of job n
w_n	Priority of job n
u_n	Buffer use of job n
R_n	Releasing time of job n
E_n	Delivery time of job n
D_n	Deadline of job n
C_n	Complete time of job n
L_n	Leave time of job n. It equals to D_n if job is expired, otherwise C_n
φ_n	It equals to 1 if job is expired, otherwise 0
S_n	Actual start time of Job n
S_{nm}	The moment when machine m is available for the processing of job n
	Indicator variables
η_{nm}	It equals to 1 if job n is processed on machine m, otherwise 0
$\mu_{nn'm}$	It equals to 1 if the process of job n precedes the process of job n' on machine m, otherwise 0
ς_{ni}	It equals to 1 if the job n is processed using the tool i, otherwise 0
$\nu_{nn'i}$	It equals to 1 if the process of job n precedes the process of job n' using tool i, otherwise 0
χ_{nj}	It equals to 1 if job n is processed by worker j, otherwise 0
$\psi_{nn'j}$	It equals to 1 if the process of job n precedes the process of job n' by worker j, otherwise 0
	Indicator functions
$\rho(n, m, \tau)$	It equals to 1 if job n is being processed on machine m on timestep τ, otherwise 0
$\phi(n, \tau)$	It equals to 1 if job n is in buffer on timestep τ, otherwise 0
$\xi(n, \tau)$	It equals to 1 if job n is in queue on timestep τ, otherwise 0
$q_1(n, m)$	It equals to 1 if job n can be processed on machine m, otherwise 0
$x_1(m, \tau)$	It equals to 1 if machine m idles on timestep τ, otherwise 0
$y_1(m, i)$	It equals to 1 if tool i can be used by machine m, otherwise 0
$y_2(i, \tau)$	It equals to 1 if tool i is available on timestep τ, otherwise 0
$z_1(m, j)$	It equals to 1 if worker j can operate machine m, otherwise 0
$z_2(j, \tau)$	It equals to 1 if worker j is available on timestep τ, otherwise 0
	Constraint variables
$b(\tau)$	The rest buffer space on timestep τ
$q(n, m, \tau)$	Being larger than 0 means job n can be processed by machine m on timestep τ; otherwise it cannot be processed.

(*continued*).

Table A1. Continued.

Symbol	Description
$x(m,\tau)$	Being larger than 0 means machine m is available on timestep τ; otherwise it is not available.
$y(m,i,\tau)$	Being larger than 0 means machine m can use tool i on timestep τ; otherwise machine m cannot use tool i on timestep τ
$z(m,j,\tau)$	Being larger than 0 means worker j can operate machine m on timestep τ; otherwise the worker j cannot operate machine m on timestep τ
	Other parameters and functions
H	A large enough positive number
$\mathbb{I}(\bullet)$	It equals to 1 if • is true, otherwise 0

St:

$$C_n = S_n\eta_{nm} + p_n, \forall n = 1,2,\ldots,N; \quad m = 1,2,\ldots,M; \tag{A2}$$

$$L_n \le D_n, \quad \forall n = 1,2,\ldots,N; \tag{A3}$$

$$S_{n'}\varsigma_{n'i} + H(1-\nu_{nn'i}) \ge S_n\varsigma_{ni} + p_n,$$
$$\forall n,n' = 1,2,\ldots,N; i = 1,2,\ldots,I; \tag{A4}$$

$$S_{n'}\eta_{n'm} + H(1-\mu_{nn'm}) \ge S_n\eta_{ni} + p_n,$$
$$\forall n,n' = 1,2,\ldots,N; m = 1,2,\ldots,M; \tag{A5}$$

$$S_{n'}\chi_{n'j} + H(1-\psi_{nn'j}) \ge S_n\chi_{ni} + p_n,$$
$$\forall n,n' = 1,2,\ldots,N; j = 1,2,\ldots,J; \tag{A6}$$

$$b(\tau) = U - \sum_n^N \sum_m^M \rho(n,m,\tau)u_n - \sum_n^N \phi(n,\tau)u_n \tag{A7}$$

$$y(m,i,\tau) = y_1(m,i)y_2(i,\tau),$$
$$\forall m = 1,2,\ldots,M; i = 1,2,\ldots,I; \tag{A8}$$

$$z(m,j,\tau) = z_1(m,j)z_2(j,\tau),$$
$$\forall m = 1,2,\ldots,M; j = 1,2,\ldots,J; \tag{A9}$$

$$x(m,\tau) = x_1(m,\tau)\left(\sum_i^I y(m,i,\tau)\right)\left(\sum_j^J z(m,j,\tau)\right),$$
$$\forall m = 1,2,\ldots,M; \tag{A10}$$

$$q(n,m,\tau) = q_1(n,m)x(m,\tau)(b(\tau) - u_n),$$
$$\forall n = 1,2,\ldots,N; m = 1,2,\ldots,M; \tag{A11}$$

$$\sum_n^N \xi(n,\tau) \le K \tag{A12}$$

$$S_n \ge R_n, \quad \forall n = 1,2,\ldots,N; \tag{A13}$$

$$S_{nm} \ge \tau \cdot \mathbb{I}(x(n,m,\tau) > 0),$$
$$\forall n = 1,2,\ldots,N; m = 1,2,\ldots,M \tag{A14}$$

Equation (A2) is the complete time of job n; Equation (A3) means the deadline should be no earlier than leave time; Equations (A4), (A5) and (A6) are used to describe the precedence constraints about machines, tools, workers indicating that each operation can be executed only when its precedence operation was finished; Equation (A7) indicates the rest buffer space on timestep τ; Equations (A8), (A9), (A10) and (A11) are the defined constraint variables, whose actual meanings are in Table A9; Equation (A12) means the total number of jobs in wait should be less than the maximum queue length K; Equation (A13) means the actual process starting time should be no earlier than the release time; Equation (A14) indicates the time that job n can start to be processed on machine m.